# N1: An Overview of Computer Architecture

CS 4290/6290; ECE 4100/6100

TM Conte

*Course notes and slides are*
*Copyright © Thomas M. Conte, 2010-2021*



1

## Overview of Topics in This Course

1. Measuring Performance
2. Memory
3. Processor design (microarchitecture)
   1. Simple Pipelining
   2. Branch prediction
   3. Complex Pipelining and Instruction-Level Parallelism (ILP)
      - Superscalar, VLIW, and vector processors
5. Multiprocessors
   - Cache coherence
   - Synchronization
   - Consistency models



5

## What is computer architecture?

- Making efficient computers
  - Done. Full stop.
- What's "efficiency?"
  - Fast (sure)
  - Low power dissipation
  - Low energy usage
  - Small size
  - Physics
- We will mostly focus on the "FAST" part in this class
  - We'll give you a menu of approaches and let you choose based on your constraints



6

## How to speed up a computer

1. Change the input: Algorithms, compilers
2. Change the logic circuits: nanoelectronics, VLSI circuits
3. Change the organization: computer architecture

Computer architecture has 2.5 (overlapping) tricks to speed up a computer:

1. Parallelism
2. Speculation
   - 2.1 With Prediction
   - 2.2 Without Prediction

Note prediction requires speculation to get performance, *but speculation does not require prediction!*



7

# Parallelism

- Parallel processing grain sizes:
  - Bit level
  - Instruction level
  - Thread level (=programmer pain)
  - Inter-Program level
- All levels except "thread level" are invisible to the programmer
- Hardware view: Flynn's classification:
  - (Single, Multiple) Instruction streams, (Single, Multiple) Data streams
  - SISD traditional single-threaded processor
  - MISD is… rare (e.g. cryptography)



8

# Example of instruction-level parallelism

A: ADD R1, R2, R3
B: ADD R4, R1, R2
C: ADD R5, R6, R4

vs.

X: ADD R1, R3, R4
Y: SUB R5, R2, R3
Z: ADD R7, R6, R4




*A, B, C will run 3x slower than X, Y, Z on this hardware: C depends on B, which in turn depends on A*



9

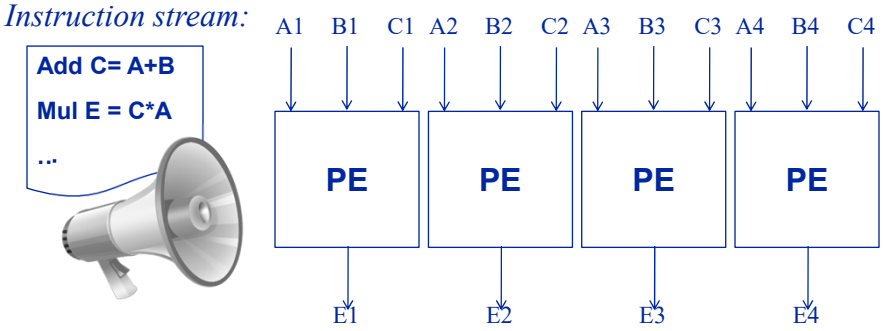
# Thread level parallelism

- Expressed as part of the program, inherent in the algorithm
- Several styles of thread level parallelism
  - Many competing taxonomies
  - Most famous is due to Mike Flynn
- Flynn Taxonomy
  - Instruction streams are either Single (one PC) or Multiple (multiple PCs)
  - Data streams are either Single (all threads work on the same data) or Multiple (each thread working on its own data



10

# Flynn's SIMD

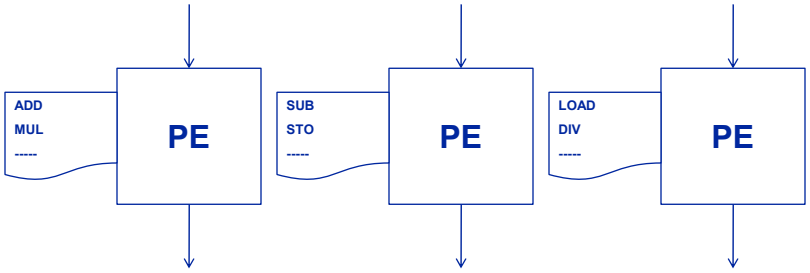



- Easy to program
  - No synchronization: everyone in "lock step"
- Not all program parallelism can be exploited with this approach



11

# Flynn's MIMD




- Each PE executes its own thread of instructions
- MIMD supports two grain sizes:
  - Program-level – no synchronization, but speedup limited to throughput
  - Thread-level – synchronization needed
    - Thread-level can "simulate" SIMD, but at a higher performance hit



12

# Known SIMD/MIMD hybrids

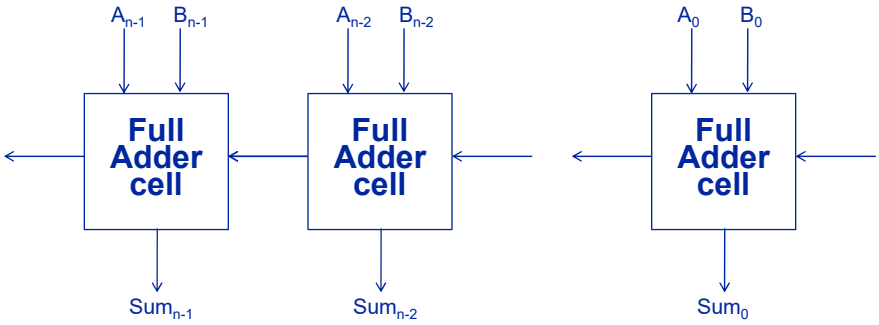
- SPMD – Single Program, Multiple Data
  - Control flow paths may vary between PEs
- SPMT – Single Program, Multiple Thread
  - New age hip name for SPMD
  - Also known as the (parallel part of) the GPU model
- GPUs?
  - All the concepts we can learn without focusing on these first
  - We will fill in the blanks before the course ends



13

## Example bit level parallelism: Ripple adder and SIMD instructions

- Ripple adder is sequential (not parallel)



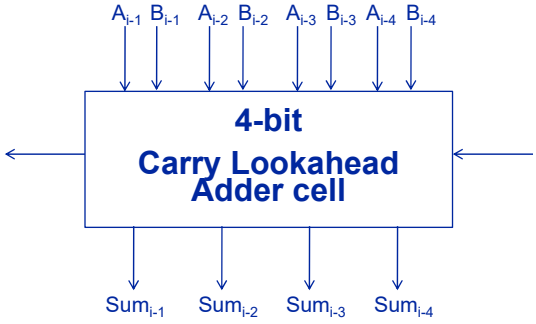

- Each full adder takes two "gate delays"
- Time for result ~ 2*wordsize

INTRO

14

## Example bit level parallelism: Carry-Lookahead Adder (CLA)

- Theorem of combinational logic: Can make any m-variable binary function with 2 levels of logic. Thus build this:




- This adder takes 2 gate delays, but many more gates
- For larger adders using CLA: time = 0.5*wordsize

INTRO

15

# Packed Word (bit level) SIMD

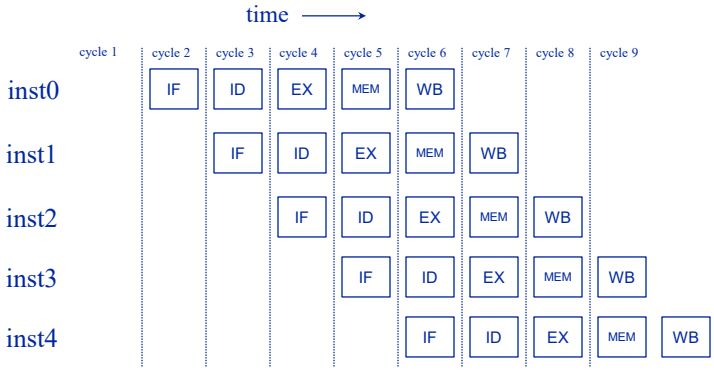
- SIMD instructions or "packed word" is an example of bit-level parallelism
- PADD.B R3,R2,R1 – with wordsize 32 bits
  - What it does: uses 4, 8-bit CLAs
  - adds R3[31:24] = R2[31:24] + R1[31:24], etc.
- What's cool here? Re-use hardware a different way
- What's not so cool: have to get the data you want into the registers "just so" (ie, aligned right) or you can't use this trick
- Many powerful extensions. Earliest was MicroUnity's, first commercial was 68040 graphics unit
- Other examples: MMX, SSE, LNI, ARM Neon etc.



16

# Next parallelism trick: Pipelining

- Isolate each of IF, ID, EX, MEM, WB with latches
- When instruction *i* is in WB, *i*+1 is in MEM, etc.
- Graphically:




*Pipelining is neither SIMD nor MIMD!*



17

# Pipelined Vector Operations

- You can accelerate execution unit processing using pipelining (e.g., the Cray-1)
- Floating-point operations take multiple steps
  - $1.23 \times 10^3 + 2.34 \times 10^{-1}$
  - First "DeNormalize" to 1230 + 0.234
  - Then ADD to get 1230.234
  - Then "ReNormalize" to get $1.230234 \times 10^3$
  - These three steps can be pipelined
- Assume adder has three stages, multiplier has three, and each "V register" is a stack of 64 values:
  - VADD V3,V1,V2
  - VMUL V5,V4,V3
- Total speedup is 6x
- This is "Cray style" vector processing (*NOT TO BE CONFUSED WITH microSIMD (packed word) vectors!)*



18

# Summary: Dimensions of parallelism

- Architecture: speculation, prediction, parallelism
  - This course is more about parallelism than prediction
- Grain size: bit-level, ILP, thread-level, program level (throughput)
  - This course is mostly about thread-level
- MIMD vs. SIMD vs. SISD (vs. MISD)
- Pipelining vs. unpipelined
- Packed word SIMD operations



21

## Moore's law means:

Computers get twice as fast every two years

*Well, that's not what Moore said……..*

26

**Moore's Law:**



**gets you 1,000 transistors today**

If you **wait two years**, then

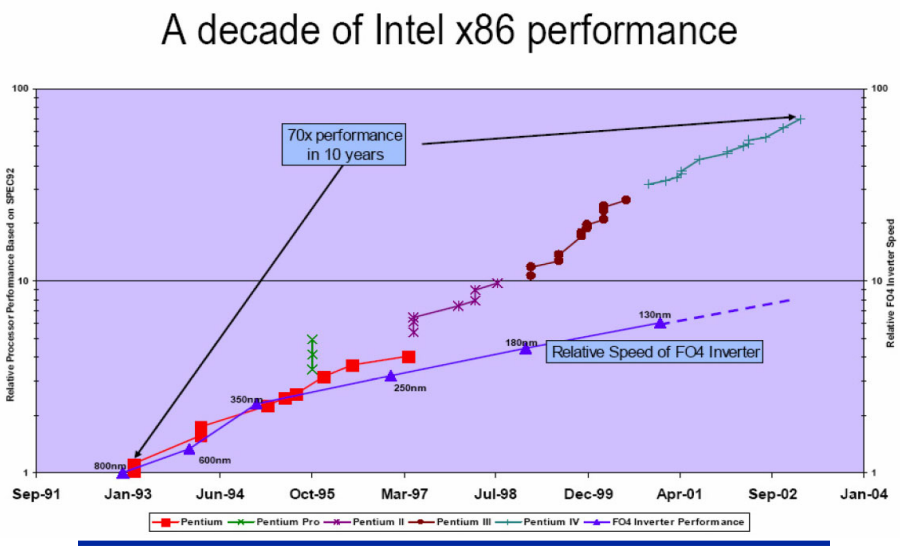
**You get 2,000 transistors for** 

***Used to track 1 to 1 with computer speed, but then…***

27

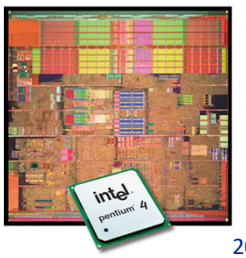
27

## Wire delays started to dominate in 1995




28

## Instruction-level parallelism scaling ended in 2005




200W/cm$^2$

### Intel Prescott

- Released Q1'05
- Could not be clocked as fast as Intel wanted
- Dissipated 200W/cm$^2$
  - Same as power density of an operating nuclear reactor core



29

## The "Power Wall" of 2005

- After Prescott, microarchitectural "tricks" could not continue to scale performance exponentially because of power constraints
- In reaction, Intel put multiple cores onto one die
  - In fact, Intel discarded the Prescott micorarchitecture and went back ten years to the P6 microarchitecture of 1995 to define the new "core microarchitecture"
- This helps with program-level parallelism, but task-level parallelism still requires changing the algorithm



30

## Summary

- Historically, performance scaled at a rate of 2x per 18-24 months
- Transistor speed stopped scaling at the historic rate in 1995
- Microarchitecture tricks to continue performance scaling took up the slack
- Microarchitecture tricks ended because of the "Power Wall" in 2005
- This caused the era of "Multicores" and "post Moore computing"



31